

Conferencia 1: lo básico de Python

Programación · Ciencia de la Computación · MatCom, Universidad de La Habana · 2026-09-16

Hoy escribimos nuestros primeros programas en Python. Al terminar la clase vas a saber escribir un programa que le pide datos al usuario, hace cuentas con ellos y muestra el resultado. Ese programa va a ser una calculadora de las raíces de una ecuación de segundo grado, y la vamos a construir pieza a pieza durante la hora.

1. Qué es un programa

Un programa es un texto con instrucciones que la computadora ejecuta en orden, de arriba hacia abajo. En Python ese texto se guarda en un archivo con extensión `.py`. Creamos el archivo `hola.py` con una sola línea:

```
print("Hola, Ciencia de la Computación")
```

y lo ejecutamos desde la terminal:

```
$ python hola.py
Hola, Ciencia de la Computación
```

Python también tiene un modo interactivo, la consola o REPL. Si escribes `python` sin ningún archivo, aparece el símbolo `>>>` y cada línea que escribas se ejecuta en el momento. La consola sirve para probar cosas pequeñas. Los programas de verdad van en archivos.

```
$ python
>>> print("Hola")
Hola
>>> exit()
```

2. Expresiones y tipos

Una expresión es un fragmento de código que produce un valor. La consola evalúa cada expresión que escribes y muestra su valor, así que funciona como calculadora. Antes de mirar cada resultado, intenta adivinarlo.

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> 2 ** 10
1024
```

Los operadores aritméticos de Python son estos:

OPERADOR	OPERACIÓN	EJEMPLO	RESULTADO
+	suma	17 + 5	22
-	resta	17 - 5	12
*	multiplicación	17 * 5	85
/	división	17 / 5	3.4
//	división entera	17 // 5	3
%	resto de la división	17 % 5	2
**	potencia	17 ** 2	289

La precedencia es la de las matemáticas. ** va primero, después *, /, // y %, y al final + y -. Si tienes dudas, usa paréntesis.

Tipos

Todo valor tiene un tipo. Por ahora nos interesan tres:

- `int`, los números enteros: 7, -3, 0.
- `float`, los números reales con punto decimal: 3.5, -0.25, 10.0.
- `str`, el texto (en inglés string), siempre entre comillas: "hoLa", '7'.

La función `type` dice el tipo de un valor:

```
>>> type(7)
<class 'int'>
>>> type(7 / 1)
<class 'float'>
>>> type("7")
<class 'str'>
```

Hay tres detalles que conviene ver con calma.

La división / siempre devuelve un `float`, aunque la división sea exacta. `10 / 2` vale `5.0` y no `5`. Si quieres un entero, usa `//`.

Los enteros de Python no tienen límite de tamaño. `2 ** 100` da `1267650600228229401496703205376` sin ningún problema.

Los `float` son aproximados, porque la computadora guarda los números en binario y `0.1` no tiene representación exacta en binario:

```
>>> 0.1 + 0.2
0.30000000000000004
```

Por eso nunca se compara si dos `float` son exactamente iguales. Lo retomaremos más adelante en el curso.

Los operadores también funcionan con texto, con otro significado. `+` concatena y `*` repite:

```
>>> "Hola, " + "mundo"
'Hola, mundo'
>>> "ja" * 3
'jajaja'
```

3. Instrucciones y variables

Una instrucción es una orden completa: hace algo, pero no necesariamente produce un valor.

`print("Hola")` es una instrucción. Un programa es una secuencia de instrucciones que se ejecutan una detrás de otra.

La diferencia entre la consola y un archivo importa aquí. En la consola, si escribes la expresión $2 + 3$, ves 5. Si pones esa misma línea sola dentro de un archivo `.py`, Python la calcula y no muestra nada. En un programa, lo que quieras ver hay que imprimirlo con `print`.

Variables

Una variable es un nombre que guarda un valor. Se crea con una asignación:

```
a = 1
b = -3
c = 2
discriminante = b ** 2 - 4 * a * c
print(discriminante)
```

1

El signo `=` significa "calcula lo que está a la derecha y guárdalo con el nombre de la izquierda". No es la igualdad de las matemáticas.

Un programa se ejecuta en el tiempo

Las instrucciones se ejecutan una a una, y cada variable guarda su valor actual, el que recibió en la última asignación. Mira este programa y decide, antes de seguir leyendo, si imprime 10 o 20:

```
a = 5
b = a * 2
a = 10
print(b)
```

Para responder, seguimos la memoria paso a paso. Cada fila de la tabla muestra el valor de cada variable después de ejecutar esa instrucción, y la casilla en negrita es la que cambió:

INSTRUCCIÓN EJECUTADA	a	b
a = 5	5	no existe
b = a * 2	5	10
a = 10	10	10
print(b)	10	10

En la segunda fila, Python lee el valor actual de `a`, que es 5, calcula $5 * 2$ y guarda 10 en `b`. La expresión `a * 2` sirve para calcular ese valor y después se descarta. En `b` queda el número 10 y nada más: `b` no recuerda que salió de `a`. Por eso, cuando `a` cambia en la tercera fila, `b` sigue valiendo 10.

```
a = 5
b = a * 2
a = 10
print(b)
```

10

Si = fuera una igualdad matemática, $b = a * 2$ tendría que seguir siendo cierta después de cambiar `a`, y el programa imprimiría 20. Es una asignación: ocurre una vez, en un momento del tiempo, con los valores que había en ese momento.

Con esa idea, esta línea tiene sentido en Python aunque no lo tenga en álgebra:

```
contador = 10
contador = contador + 1
print(contador)
```

11

INSTRUCCIÓN EJECUTADA	contador
contador = 10	10
contador = contador + 1	11
print(contador)	11

La segunda línea lee el valor actual de `contador` (10), le suma 1 y guarda el resultado (11) en la misma variable. El valor viejo se pierde.

Las reglas para nombrar variables son pocas. El nombre puede tener letras, dígitos y `_`, no puede empezar con un dígito y no puede ser una palabra reservada de Python como `if` o `for`. Python distingue mayúsculas de minúsculas, así que `area` y `Area` son dos variables distintas. La costumbre en Python es escribir los nombres en minúsculas y separar las palabras con `_`, como en `precio_total`.

Usar una variable que todavía no existe es un error:

```
print(radio)
```

```
$ python radio.py
Traceback (most recent call last):
  File "radio.py", line 1, in <module>
    print(radio)
    ^^^^^
NameError: name 'radio' is not defined
```

Lee el mensaje de error de abajo hacia arriba. La última línea dice qué pasó (`NameError`, el nombre `radio` no está definido) y las de arriba dicen dónde.

4. Escribir en la consola

print recibe uno o más valores, los escribe separados por un espacio y termina con un salto de línea:

```
x = 7
print("x vale", x, "y su cuadrado es", x ** 2)
```

```
x vale 7 y su cuadrado es 49
```

Dos parámetros opcionales cambian ese comportamiento. sep es el separador entre valores y end es lo que se escribe al final:

```
print(2026, 9, 16, sep="- ")
print("sin salto de línea", end=" ")
print("y seguimos en la misma línea")
```

```
2026-9-16
sin salto de línea y seguimos en la misma línea
```

f-strings

La forma más cómoda de mezclar texto y valores es un f-string: un texto con una f delante de las comillas, donde todo lo que va entre llaves se evalúa y se inserta.

```
a = 1
b = -3
c = 2
print(f"La ecuación es {a}x2 + {b}x + {c} = 0")
print(f"El discriminante vale {b ** 2 - 4 * a * c}")
```

```
La ecuación es 1x2 + -3x + 2 = 0
El discriminante vale 1
```

Dentro de las llaves va cualquier expresión. Después de : se puede indicar el formato; .2f significa "con dos cifras decimales":

```
precio = 1234.5678
print(f"Total: {precio:.2f} pesos")
```

```
Total: 1234.57 pesos
```

5. Leer de la consola

input detiene el programa, espera a que el usuario escriba una línea y presione Enter, y devuelve lo que escribió. Si le pasas un texto, lo muestra como pregunta.

```
nombre = input("¿Cómo te llamas? ")
print(f"Hola, {nombre}")
```

```
$ python saludo.py
¿Cómo te llamas? Ana
Hola, Ana
```

Probemos ahora un programa que suma dos números. Antes de ejecutarlo, piensa qué va a imprimir si el usuario escribe 2 y 3.

```
x = input("Primer número: ")
y = input("Segundo número: ")
print(f"La suma es {x + y}")
```

```
$ python suma.py
Primer número: 2
Segundo número: 3
La suma es 23
```

El programa imprime 23, porque input siempre devuelve un str, aunque el usuario escriba dígitos. x vale "2", y vale "3", y + entre dos textos los concatena.

Conversiones

Para operar con los números hay que convertir el texto. Cada tipo tiene una función con su mismo nombre que convierte a ese tipo:

```
print(int("42") + 1)
print(float("2.5") * 2)
print(str(42) + "!")
print(int(3.9))
print(int(-3.9))
```

```
43
5.0
42!
3
-3
```

int aplicado a un float descarta la parte decimal: no redondea, corta hacia el cero. El programa de la suma queda así:

```
x = int(input("Primer número: "))
y = int(input("Segundo número: "))
print(f"La suma es {x + y}")
```

La conversión falla si el texto no representa un número del tipo pedido. int("3.5") también falla, porque "3.5" no es un entero escrito en texto:

```
>>> int("3.5")
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    int("3.5")
```

```
~~~~~  
ValueError: invalid literal for int() with base 10: '3.5'
```

```
>>> int("hola")  
Traceback (most recent call last):  
  File "<python-input-1>", line 1, in <module>  
    int("hola")  
~~~~~  
ValueError: invalid literal for int() with base 10: 'hola'
```

Si el usuario puede escribir decimales, convierte con `float`. Y sumar un texto con un número sin convertir da otro error, esta vez de tipo:

```
>>> "5" + 3  
Traceback (most recent call last):  
  File "<python-input-2>", line 1, in <module>  
    "5" + 3  
~~~~~  
TypeError: can only concatenate str (not "int") to str
```

6. La biblioteca `math`

Python trae una biblioteca estándar con muchos módulos listos para usar. Las funciones matemáticas están en el módulo `math`, que hay que importar antes de usarlo:

```
import math  
  
print(math.sqrt(16))  
print(math.pi)  
print(math.floor(3.7), math.ceil(3.2))
```

```
4.0  
3.141592653589793  
3 4
```

NOMBRE	QUÉ HACE	EJEMPLO	RESULTADO
<code>math.sqrt(x)</code>	raíz cuadrada	<code>math.sqrt(2)</code>	1.4142135623730951
<code>math.pi</code>	la constante π	<code>math.pi</code>	3.141592653589793
<code>math.floor(x)</code>	mayor entero menor o igual que x	<code>math.floor(-3.7)</code>	-4
<code>math.ceil(x)</code>	menor entero mayor o igual que x	<code>math.ceil(3.2)</code>	4
<code>math.sin(x), math.cos(x)</code>	seno y coseno, x en radianes	<code>math.cos(0)</code>	1.0

Dos funciones parecidas no necesitan `import`, porque son parte del lenguaje: `abs(-5)` da 5, y `round(3.14159, 2)` da 3.14.

Compara `math.floor(-3.7)`, que da -4, con `int(-3.7)`, que da -3. `floor` baja siempre y `int` corta hacia el cero.

La calculadora completa

Ya tenemos todas las piezas. Las raíces de $ax^2 + bx + c = 0$ son

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

y el programa es la fórmula escrita en Python:

```
import math

a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))

discriminante = b ** 2 - 4 * a * c
x1 = (-b + math.sqrt(discriminante)) / (2 * a)
x2 = (-b - math.sqrt(discriminante)) / (2 * a)

print(f"x1 = {x1}")
print(f"x2 = {x2}")
```

```
$ python raices.py
a: 1
b: -3
c: 2
x1 = 2.0
x2 = 1.0
```

Probemos ahora con $x^2 + 1 = 0$, es decir $a = 1$, $b = 0$, $c = 1$. El discriminante vale -4 y la raíz cuadrada de un número negativo no es un número real:

```
$ python raices.py
a: 1
b: 0
c: 1
Traceback (most recent call last):
  File "raices.py", line 8, in <module>
    x1 = (-b + math.sqrt(discriminante)) / (2 * a)
           ~~~~~^~~~~~
ValueError: expected a nonnegative input, got -4.0
```

En versiones de Python anteriores a la 3.14 el mensaje es `ValueError: math domain error`. Para que el programa responda "no tiene raíces reales" en vez de fallar, necesita decidir qué hacer según el valor del discriminante. Eso es lo que vemos en la próxima clase: los condicionales. Si pruebas $a = 0$, vas a encontrar otro error que también se arregla con un condicional.

7. Resumen

- Un programa es una secuencia de instrucciones que se ejecutan en orden.
- Una expresión produce un valor, y todo valor tiene un tipo: `int`, `float`, `str`.
- `=` guarda un valor en una variable; no es la igualdad matemática.
- `print` escribe en la consola, y los f-strings mezclan texto y valores.
- `input` lee de la consola y siempre devuelve un `str`. Para hacer cuentas hay que convertir con `int` o `float`.
- `import math` da acceso a `sqrt`, `pi`, `floor`, `ceil` y el resto de las funciones matemáticas.

Ejercicios

1. Escribe un programa que lea los dos catetos de un triángulo rectángulo e imprima la hipotenusa con dos cifras decimales.
2. Escribe un programa que lea una temperatura en grados Celsius y la imprima en Fahrenheit. La fórmula es $F = \frac{9}{5}C + 32$.
3. Escribe un programa que lea un número de segundos e imprima cuántas horas, minutos y segundos son. Por ejemplo, 3665 debe imprimir 1 h 1 min 5 s. Usa // y %.
4. Escribe un programa que lea el radio de un círculo e imprima su área y su perímetro.
5. Escribe un programa que lea un número entero de tres cifras e imprima la suma de sus cifras. Por ejemplo, 472 debe imprimir 13.
6. Sin ejecutarlo, di qué imprime cada línea. Después compruébalo en la consola.

```
print(17 // 5, 17 % 5, -17 // 5)
print(int("7") * 2, "7" * 2)
print(round(2.5), round(3.5))
```

Si alguno de los resultados te sorprende, busca en la documentación de Python cómo redondea round.